

GAMS assignment

Student Michelangelo Inzerilli

1 Description of the problem

The problem addressed in the proposed solution, using GAMS, is related to the home's electricity usage over a typical day, and particularly focuses on the minimization of the electricity bill. Three main parameters were taken into account:

- electricity prices during the day
- appliances used
- total running time of each appliances during the day

In fact, electricity prices vary during the day, being higher during mornings and late-afternoons, when peak-consumptions happen. Prices are determined the day before, in the so called day-ahead market, which allows the possibility of knowing them in advance. As appliances, the fridge, the dishwasher, the washer, the HVAC and the EV were selected. The choice depended on the fact that these utilities can be controlled, and don't strictly need to run/be charged at fixed times of the day. Lastly, the following total running times and energy usage of the appliances [kWh per hour] during the day were set up:

- Fridge: 24 hours, 0.8 kWh per hour [1]
- Dishwasher: 2 hours, 1.8 kWh per hour [1]
- Washer: 1 hours, 1.0 kWh per hour [2]
- HVAC: 3 hours, 2.0 kWh per hour [3]
- EV: 4 hours, 7.2 kWh per hour [1]

2 Mathematical model

Firstly, using *Set*, the t element was determined, going from 1 to 24 hours. Secondly, the input data were imported from an Excel file, representing the day-ahead electricity prices per hour [4]. The imported values were in €/MWh and collected in the $rawPrice(t)$ parameter, later converted in €/kWh and stored in $price(t)$. Afterwards, " a " was set, containing the appliances' categories (fridge, dishwasher, washer, HVAC, EV).

```
Set t /t1*t24/;

Parameter rawPrice(t), price(t);

* Define your working path
$setGlobal myPath "C:/Users/michelangeloinze/Desktop/Energy_Policy/GAMS_assignment/

* Step 1: Use.gdx to convert Excel sheet to GDX
$call.gdx.exe input="%myPath%/Day_Ahead_Electricity_Prices.xlsx" output="%myPath%/

* Step 2: Load from GDX into GAMS
$gdxin %myPath%/in.gdx
$load rawPrice
$gdxin

* Convert €/MWh → €/kWh
price(t) = rawPrice(t) / 1000;

Set
a appliances /fridge, dishwasher, washer, HVAC, ev/;
```

Figure 1: Firsts strings of the code

Secondly, using *Parameter*, the total running times and energy usage of the appliances [kWh per hour] during the day were imposed, together with a new parameter called $timewindow(a, t)$, imposed at 0 value by default or 1 using a conditional setting. Taking the dishwasher as an example, if the time of the day is between 8 am and 10

pm, $timewindow(dishwater, t)$ is changed to 1. The time slot between 8 am and 10 pm was chosen because it is retained a common time to run a dishwasher. The same decision-making process was applied for all the appliances. It was then called $x(a, t)$ as a *Binary Variable*, being 1 if the appliance a runs at time t .

```
* Time window matrix (1 if appliance 'a' is allowed to run in time 't')
Parameter timewindow(a,t);

* Default all to 0
timewindow(a,t) = 0;

* Appliance time windows using $ conditional
timewindow("fridge",t) = 1;
timewindow("dishwasher",t)$(ord(t) ≥ 8 and ord(t) ≤ 22) = 1;
timewindow("washer",t) $(ord(t) ≥ 7 and ord(t) ≤ 21) = 1;
timewindow("HVAC",t) $(ord(t) ≥ 6 and ord(t) ≤ 23) = 1;
timewindow("ev",t) $(ord(t) ≤ 7 or ord(t) ≥ 22) = 1;

* Binary decision variable: x(a,t) = 1 if appliance a runs at time t
Binary Variable x(a,t);
```

Figure 2: Second code's block

Thirdly, the objective variable $totalCost$ was called, representing the value that the model has to minimize, and in general the total daily electricity bill. Afterwards, the *Equations* were declared and then defined, and are shown in Figure 3.

```
* Objective variable: total electricity cost
Variable totalCost;

Equation
obj "Objective function"
applianceRuntime(a) "Ensure each appliance runs for required hours"
timeWindowLimit(a,t) "Only allow appliances to run within allowed time slots";

* Objective function: Minimize total cost
obj..
totalCost =e= sum((a,t), x(a,t) * power(a) * price(t));

* Each appliance must run for exactly its required runtime
applianceRuntime(a)..
sum(t, x(a,t)) =e= runtime(a);

* Respect allowed time windows
timeWindowLimit(a,t)..
x(a,t) =l= timewindow(a,t);
```

Figure 3: Third code's block

The objective function multiplies the energy used by the appliance and the electricity prices at the given time, if the appliance is running, summing all up. $applianceRuntime(a)$ ensures each appliance runs for exactly its required number of hours, while $timeWindowLimit(a, t)$ ensures appliances only run during allowed times (i.e., where $timewindow = 1$).

Finally, GAMS is told to solve the optimization model using Mixed Integer Programming (MIP), minimizing $totalCost$. $x.l$ then shows which appliances are scheduled at which times, while $totalCost.l$ shows the final total bill (the result of your optimization).

```
Model HomeEnergy /all/;

Solve HomeEnergy using mip minimizing totalCost;

* Display results
Display x.l, totalCost.l;
```

Figure 4: Fourth code's block

3 Results and suggestions for improvement

The results show the optimized organization of the appliances during the day, indicating whenever it is better, from a cost-saving perspective, to use each of them.

- Fridge: 24 hours by default
- Dishwasher: from 12 to 14
- Washer: from 13 to 14
- HVAC: from 12 to 15
- EV: from 2 to 5 and from 23 to 24

The total electricity bill calculated, using this methodology, is of 5.995€.

The developed model is simplified and does not consider various aspects such as the possible requirement of a system to continuously run for consecutive hours, completing its process, and not going ON-OFF arbitrarily. Furthermore, it does not include possible on-site energy supply, such as a PV system, which are more and more integrated in houses, and which produce only at given times of the day (when where is the sun) and which energy need to be immediately used (if there is no a storage). Moreover, the first strings of the code are designed to work with the Excel file of the website [4] used to download the Day-Ahead electricity market prices, and would need some adjustments to work with other inputs.

Concluding the model is working with the simplified given information, but would need optimization to be integrated, for example, in an automatized process.

References

- [1] Schneider Electric Blog. *How much electricity does a fridge, dishwasher, and an electric car use? We did the math.* 2023. URL: <https://blog.se.com/sustainability/2023/01/24/how-much-electricity-does-a-refrigerator-dishwasher-and-an-electric-car-use-we-did-the-math/>.
- [2] EnergySage. *How many watts does a washing machine use?* 2024. URL: <https://www.energysage.com/electricity/house-watts/how-many-watts-does-a-washing-machine-use/>.
- [3] EnergySage. *How much energy does a heat pump use?* Taking 2,000 W as reference. 2024. URL: <https://www.energysage.com/electricity/house-watts/how-many-watts-does-an-air-source-heat-pump-use/>.
- [4] GME. *Electricity Day_aheadmarket.* 2025. URL: <https://www.mercatoelettrico.org/en-us/Home/Results/Electricity/MGP/Results/PUN#IntestazioneGrafico>.

Appendix

Another .gms version of the file, called "Version2", contains a different method to input the electricity prices. In this version, the electricity prices are imported using a .inc file, containing the prices in €/kWh, rather than the Excel one directly downloaded from the cited website [4]. This was done because on macOS is not possible to run the *gdxxrw.exe* command.

```
1 Set t /t1*t24/;  
2 Parameter price(t);  
3  
4 $include priceData.inc  
5
```

Figure 5: Version2 importation code's block

All the utilized files are included in the submission .zip file, which include a folder called "model" where they are stored. This folder includes the two .gms models, one using the *gdxxrw.exe* command, the other one using the .inc file, together with the Excel file electricity prices.